

GDFuzz: An efficient directed fuzzing method based on XAI[☆]Junru Li[✉], Zhongxu Yin^{✉*}, Guoxiao Zong, Haiya Sang[✉]

Information Engineering University, Zhengzhou, 450000, China

ARTICLE INFO

Keywords:

Fuzzing
Vulnerability detection
Open source software
Explainable deep learning

ABSTRACT

As software systems grow increasingly complex, traditional fuzz testing struggles with inefficiencies, limited coverage, and poor adaptability to large-scale codebases with intricate input formats. These challenges are exacerbated in directed fuzz testing, where a lack of explainability often hampers effective user intervention and fine-grained input control, limiting both manageability and precision. To address these limitations, this paper introduces GDFuzz, an innovative fuzz testing framework that enhances directed testing by integrating explainable deep learning and interactive controllability. At its core, GDFuzz leverages the Local Interpretable Model-Agnostic Explanations (LIME) technique to identify and protect critical input bytes that influence the execution path to target locations. This mechanism, termed Sample Masks, guides mutation strategies toward more effective and targeted exploration. Furthermore, GDFuzz provides a real-time visual intervention interface, enabling users to dynamically adjust strategies based on neural network outputs, bridging the gap between automated testing and human expertise. Additionally, GDFuzz integrates a Dual-Queue Cyclic Simulated Annealing (DQCSA) algorithm and adaptive incremental learning to adapt to our explanation mechanism. We validate GDFuzz through comparative experiments on five real-world applications against four state-of-the-art fuzzers: AFL, AFL++, SelectFuzz, FormatFuzzer and AFLGo. Results demonstrate that GDFuzz improves effective execution rates by an average of 185% across all benchmarks, highlighting the impact of its interpretability-driven enhancements. Notably, GDFuzz uncovered eight previously undisclosed vulnerabilities (0-days) in widely used open-source libraries, with four assigned CVE identifiers and four pending review. These achievements underline the value of explainable and controllable testing in advancing directed fuzzing methodologies.

1. Introduction

Fuzz Testing, as an automated security testing method, leverages randomized input mutations to quickly identify potential vulnerabilities and anomalous behavior, establishing itself as a cornerstone of software security assurance. However, in scenarios involving large-scale codebases and complex input formats, existing tools often struggle to effectively guide inputs toward deep target functions, leading to inefficient resource usage and limited vulnerability discovery capabilities.

Directed fuzz testing tools, such as AFLGo (Böhme et al., 2017), were introduced to address these challenges (Luo et al., 2023; Chen et al., 2018; Huang et al., 2022). By employing heuristic strategies, such as path distance-based analysis, combined with static analysis and runtime path information, these tools direct inputs toward specific target functions, significantly enhancing testing efficiency. Building upon this foundation, numerous AFL-based tools have been developed to refine directed fuzzing approaches and accelerate their evolution.

Despite recent advancements in directed fuzz testing, two critical issues persist in the mutation strategy.

Limited correlation between samples and target code locations. Mainstream directed fuzz testing tools (Böhme et al., 2017; Du et al., 2022; Lee et al., 2021) typically rely on path-guidance algorithms. These algorithms combine static analysis with dynamic instrumentation to calculate the “path distance” between the input and the target function, optimizing seed scheduling and mutation strategies accordingly. During testing, these distances are dynamically applied to prioritize exploration around the target path. However, this approach mainly adjusts the sample scheduling algorithm and does not provide guidance for mutation operators or mutation positions in the sample mutation algorithm. This results in blind exploration during fuzzing: the fuzzer randomly selects positions and applies mutation operators without specific guidance. Empirical studies (Böhme et al., 2020) have shown that vulnerabilities are only triggered when the fuzzer reaches the vulnerable code locations. Therefore, the efficiency of testing is

[☆] Editor: Aldeida Aleti.

* Corresponding author.

E-mail addresses: du4tli@outlook.com (J. Li), yinzhu@163.com (Z. Yin), zonegoalshall@foxmail.com (G. Zong), hys_79591646@outlook.com (H. Sang).

thus significantly influenced by the proportion of effective executions. In this paper, we define effective execution rate as executions that successfully reach the target code or expose vulnerabilities relative to the total number of test executions. In recent years, researchers have proposed various improvements. For instance, tools like *katch* (Marinescu and Cadar, 2013) and *QYSM* (Yun et al., 2018) integrate symbolic execution to generate inputs that satisfy syntactic constraints, followed by deep exploration through fuzzing (Liang et al., 2019). While these approaches substantially improve path coverage in complex scenarios, their high performance overhead and path explosion remain critical bottlenecks. Moreover, other efforts have used machine learning techniques for input format correction and feature extraction (Godefroid et al., 2017; Zakeri Nasrabadi et al., 2021; Shi et al., 2023). These tools utilize models to learn the syntax and semantic features of complex inputs, achieving impressive results in specific scenarios. However, the “black-box” nature of deep learning models limits the interpretability of their path-guidance or sample-repair strategies, complicating debugging for users.

Lack of Balancing between Exploration and Exploitation. Directed fuzz testing aims to prioritize inputs that bring the execution closer to specific target locations. However, this prioritization requires carefully balancing exploration—discovering new execution paths—and exploitation—focusing testing efforts on paths that are near the target. Existing approaches often employ simulated annealing algorithms to navigate this trade-off (Xiang et al., 2024; Luo et al., 2023; Lin et al., 2020; Zhang et al., 2024b). These algorithms attempt to dynamically adjust the focus between exploration and exploitation, typically starting with an emphasis on exploration and gradually shifting toward exploitation as progress is made. However, many of these methods rely on rigid, predefined strategies for transitioning between the two phases, which fail to adapt in real-time to feedback from the fuzzing process. This lack of flexibility can lead to significant inefficiencies—such as excessive exploration of redundant paths or premature convergence on limited paths during exploitation. As a result, critical execution paths may be overlooked. Furthermore, traditional fuzzing tools often rely on simple metrics like control flow distance to prioritize input samples. While this is effective for path coverage, it can lead to clustering around a narrow set of control flow paths, missing the broader space of potential execution paths. More importantly, existing tools lack mechanisms to encourage the diversity of test cases across both control flow and data flow, limiting their ability to uncover complex vulnerabilities that depend on specific path conditions.

Insufficient integration of human expertise in the fuzzing loop. Users’ intuition about test samples, while abstract, provides strong guidance. In current fuzzing practices, fuzzers are often treated as “black boxes”, requiring minimal configuration by users. However, whether vulnerabilities can be triggered remains beyond the users’ control. Users often lack visibility into the issues during fuzzing, the points where execution is blocked, and the underlying reasons for such blocks. Current research on the explainability of fuzzing often focuses on superficial aspects. For example, *OSS-Fuzz* introduced the Fuzz Introspector framework (OSS-Fuzz, 2022), which leverages LLVM’s PGO mechanism to provide real-time coverage feedback during testing, offering users a basic understanding of coverage progress. However, users cannot interpret and intervene effectively at the sample level. Methods, such as *DDGF* (Fang et al., 2024), attempt to improve directed fuzzing efficiency by leveraging human intuition to select target paths. However, these approaches primarily conduct shallow feedback in fuzzing and fail to delve into deeper level of the core algorithms. As a result, the human action in influencing and enhancing the fuzzing process remains significantly constrained.

Our Approach. In this paper, we propose *GDFuzz*, a lightweight fuzzing framework designed to establish sample-to-target code correlation awareness while incorporating human intuition into the fuzzing process. *GDFuzz* achieves this through a lightweight parallel mechanism linking sample bytes with target code locations, as well as optimizing the mutation strategy in directed fuzz testing.

Specifically, *GDFuzz* operates by collecting samples during testing and categorizing them into positive samples (those reaching the target code region) and negative samples (those failing to reach it). These samples are then fed into a convolutional neural network (CNN)-based binary classification model. Leveraging the model’s deep feature detection capabilities, *GDFuzz* rapidly and accurately identifies the correlation between specific sample bytes and the target code region. Subsequently, *GDFuzz* employs the Local Interpretable Model-Agnostic Explanations (LIME) framework (Ribeiro et al., 2016) to interpret the trained deep learning model. This enables the extraction of the key relationships between sample bytes and target code locations, allowing the generation of sample masks that protect highly correlated bytes from mutation. By preventing mutations in these critical regions, *GDFuzz* significantly enhances the rate of effective executions aimed at the target code.

Additionally, *GDFuzz* extends this correlation-awareness mechanism to facilitate human-machine collaboration. Sample masks are visualized and presented to users, who can interactively modify these masks in real time, thereby fine-tuning the mutating direction and improving overall efficiency.

To address the imbalance between exploration and exploitation phases in fuzz testing, we introduce a Dual-Queue Cyclic Simulated Annealing Algorithm, which partitions samples into exploration and exploitation queues. By employing a dynamic scheduling strategy, this algorithm balances the requirements of path exploration and targeted testing, thereby accommodating both control flow and data flow dependencies essential for triggering vulnerabilities. Moreover, to mitigate the potential decline in model accuracy caused by changes in sample diversity during testing, we propose an Adaptive Incremental Learning Algorithm. This approach dynamically adjusts the training process based on sample diversity metrics, striking a balance between computational overhead and model accuracy.

We evaluated *GDFuzz* on benchmarks such as *Magma* and real-world software systems, demonstrating its significant performance improvements compared to state-of-the-art directed fuzzing tools. Notably, *GDFuzz* exhibited up to a 185% increase in effective execution rates and identified 10 previously undisclosed vulnerabilities (7 assigned CVEs).

This paper makes the following key contributions:

- A novel **Keyword Byte Awareness Mechanism** that enhances mutation precision through byte-to-target code correlation analysis.
- A **Dual-Queue Cyclic Simulated Annealing Algorithm** that balances exploration and exploitation in directed fuzzing, addressing inefficiencies in traditional mutation strategies.
- An **Adaptive Incremental Learning Framework** that dynamically updates its mutation guidance models based on real-time sample diversity assessments.
- Comprehensive implementation and evaluation of **GDFuzz**, achieving state-of-the-art performance on both benchmark datasets and real-world applications.

2. Background and motivation

2.1. Explainable fuzzing

Computers excel over humans in terms of computational speed and accuracy, whereas humans possess superior intuition and insight. Traditional fuzz testing lacks the ability to understand program semantics and sequence, which limits its effectiveness. To address the black-box nature of fuzz testing, numerous studies have introduced interpretability interfaces to bridge the gap in fuzzers’ awareness of program semantics and sequence (Wan et al., 2024; Yan et al., 2023; Bundt et al., 2023; Zhou et al., 2019).

For instance, Fuzz Introspector (OSS-Fuzz, 2022) aggregates information such as coverage and execution counts, providing developers

with an overview of their fuzzing progress. This tool helps users identify bottlenecks and obstacles in the fuzzing process and offers interfaces for users to intervene. Similarly, hmfuzz (Bundt et al., 2023) provides additional feedback by identifying the basic blocks that contribute most significantly to coverage improvement, enabling users to focus on critical obstacles and accelerate coverage gains.

However, existing methods predominantly focus on shallow feedback, such as coverage metrics, to guide the fuzzing process while offering little or no explanation of the mutation strategies used. Mutation strategies are the core of fuzz testing, and their effectiveness directly determines the overall efficiency of the fuzzer. The lack of interpretability at this deeper level leaves room for improvement in both understanding and optimizing fuzzing processes.

2.2. Explainable deep learning

In recent years, deep learning (DL) techniques have been increasingly applied in the field of software testing, particularly in fuzz testing, to enhance testing efficiency and code coverage (Wang et al., 2019; Zong et al., 2020). Researchers aim to leverage the automatic feature extraction capabilities of deep neural networks (DNNs) to identify potential crash-inducing input patterns from large amounts of test data, thereby accelerating vulnerability detection.

While deep learning methods have proven effective in optimizing input generation and path exploration, their inherent “black-box” nature remains a significant challenge. Specifically, traditional fuzz testing tools rely on coverage feedback, but using DNNs to generate inputs and guide testing often lacks interpretable feedback mechanisms. This opacity in decision-making processes can affect the reliability of testing results. To address this issue, researchers have proposed various explainable deep learning techniques to improve model transparency and interpretability (Wong et al., 2024; Gulmez et al., 2024). For instance, methods based on attention mechanisms and feature importance analysis have enhanced understanding of DNN decision-making processes to some extent. However, the application of these techniques in fuzz testing is still in its exploratory phase. Existing research has primarily focused on using feature heatmaps to explain the critical features of model predictions (Shi et al., 2022; Zhu et al., 2023). Nevertheless, these methods face challenges in software testing because they cannot be directly translated into strategies to guide testing.

Moreover, many DL-based fuzz testing approaches emphasize input pattern recognition rather than optimizing input format repair and path exploration, limiting their ability to significantly improve testing efficiency.

2.3. Existing mutation scheduling algorithms

Most existing directed fuzz testing tools adopt simulated annealing algorithms to balance the scheduling of samples during the exploration and exploitation phases. The core idea of these algorithms is to allocate energy—defined as computational effort or mutation intensity—to test samples based on a combination of factors such as path distance, execution time, and testing progress. By dynamically adjusting the energy assigned to inputs, these algorithms strive to either prioritize exploring new paths (exploration) or refine inputs closer to the target region (exploitation). In the context of distance calculation, these tools typically employ control flow-centric metrics, often based on basic block coverage. For instance, AFLGo (Böhme et al., 2017) calculates the “path distance” between a given input and the target function by analyzing control flow graphs (CFGs) obtained through static analysis. This approach effectively reduces the computational complexity of distance evaluation and provides a straightforward mechanism to guide mutation scheduling.

However, this reliance on control flow-based distance metrics introduces a critical limitation. While control flow conditions are necessary for reaching target locations, they are insufficient for triggering many

vulnerabilities. In practice, vulnerability triggering often requires satisfying additional data flow conditions, such as specific input values, memory state constraints, or inter-variable dependencies. The exclusive focus on control flow metrics neglects these data flow requirements, resulting in a significant gap between reaching the target location and successfully triggering a vulnerability.

2.4. Motivation

Modern software systems are becoming increasingly complex, characterized by intricate control flows and stringent input validation processes. These complexities pose significant challenges to traditional testing techniques. While fuzz testing has emerged as a cornerstone for ensuring software security and reliability, its ability to systematically explore deep program paths remains limited. This limitation is particularly evident when testing software components with strict input format requirements, such as XML parsers or cryptographic libraries.

```

1  Int xmlParseElementDecl(xmlParserCtxtPtr
   ctxt) {
2  if ((CUR != '<') || (NXT(1) != '!'))
3      return(ret);
4      SKIP(2);
5  if (CMP7(CUR_PTR, 'E', 'L', 'E', 'M',
   'E', 'N', 'T')) {
6      // buffer overflow in here...
7  }
8  if (CMP5(CUR_PTR, 'E', 'M', 'P', 'T',
   'Y')) {
9      ...
10 }
11 ...

```

Listing 1: Motivating Example

Traditional fuzzing tools typically rely on random or heuristic-driven input mutation strategies, which often fail to generate semantically valid inputs capable of traversing complex execution paths or triggering specific vulnerabilities. Consider the code snippet in Listing 1, which demonstrates a stack-overflow vulnerability in the libxml2 library, a widely used XML document parser developed by GNOME. From a human perspective, it is evident that the vulnerable execution path can only be reached if the XML document contains specific *ELEMENT* keywords. To efficiently reproduce this vulnerability, a natural approach would involve preserving the *ELEMENT*-related fields in input samples. However, traditional fuzzing tools lack the ability to establish a correlation between input content and the target vulnerability location. Furthermore, they are unable to analyze the code to determine which specific data elements are missing in the input sample, making it difficult to generate inputs capable of efficiently triggering the vulnerability.

```

1  Int DivideZero(int x, int y) {
2  int z;
3  if (x > 10){
4      x = x - 10;
5  }
6  else{
7      if (y > 10)
8          y = y - 10;
9      x = x - 5;
10 }
11 z = y / x;
12 return z;
13 }
14 ...

```

Listing 2: Code snippet which has Divide by Zero vulnerability

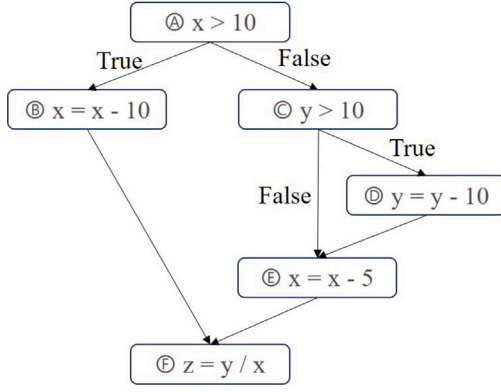


Fig. 1. Control-flow graph of the motivating example.

Another critical limitation of existing directed fuzzing tools lies in their inability to promote diverse exploration of control flow paths, resulting in an incomplete representation of critical execution paths. As shown in listing 2 and Fig. 1, consider a scenario where a vulnerability involves a shortest basic block call chain represented as $A \rightarrow C \rightarrow E \rightarrow F$. Current directed fuzzers, which prioritize samples based on basic block distance metrics, are more likely to focus on samples following a call chain like $A \rightarrow B \rightarrow F$. When the fuzzer identifies a test case that reaches basic block F , it tends to prioritize mutations within F , leading to local exploration. This narrow focus often neglects the impact of broader control flow diversity and its interplay with data flow, which are crucial for triggering certain vulnerabilities.

These limitations emphasize the need for mechanisms that empower users to guide directed fuzzing tools while maintaining the structural integrity of input samples. For example, once an input satisfies specific requirements, such as including the *ELEMENT* field for libxml2, the fuzzing tool should associate this input with its corresponding code location and protect critical bytes from mutation. Although traditional directed fuzzers adjust energy levels based on proximity to the target location, they fail to distinguish between “close” and “distant” samples in terms of mutation strategy effectiveness. Consequently, the mutation process remains largely probabilistic—vulnerabilities are triggered only when the right mutation operator happens to modify the right position. This randomness reduces fuzzing efficiency and effectiveness. At the same time, users also require an approach capable of automatically increasing control flow diversity during testing, especially when triggering vulnerabilities proves difficult. By ensuring broader exploration of critical execution paths, such a mechanism could significantly improve the chances of vulnerability detection.

3. GDFuzz

In this section, we focus on explaining the design principles underlying the implementation of GDFuzz. Fig. 2 illustrates an overview of GDFuzz. We designed a dual-protection mechanism to achieve the perception and preservation of the association between input samples and target code regions.

3.1. An overview of GDFuzz

GDFuzz builds on AFLGo by incorporating directed fuzzing with feature-aware learning, dynamic scheduling, and interactive explainability. As illustrated in Fig. 2, the system operates in a closed-loop pipeline composed of execution monitoring, learning, mutation, and adaptive scheduling. Seed inputs are executed and classified into positive or negative samples based on whether they reach the target region. These labeled samples feed into a CNN-based classifier that is

continuously refined through incremental learning, enabling the system to adapt as fuzzing progresses.

To make the learning process interpretable and mutation more effective, GDFuzz integrates an explanation module using LIME to identify key input bytes correlated with target reachability. This yields fine-grained mutation masks that preserve critical semantics during input generation. The mutation engine then leverages these masks while coordinating samples through a Dual-Queue Cyclic Simulated Annealing strategy: inputs are dynamically scheduled between a priority (exploitation) queue and a low-priority (exploration) queue, maintaining a balance between deep exploration and focused path execution.

As fuzzing progresses and sample diversity increases, GDFuzz uses a MinHash-based similarity metric to determine whether new samples are sufficiently novel to trigger classifier updates, ensuring a balance between model freshness and computational overhead. To further enhance flexibility, GDFuzz introduces a human-in-the-loop interface, allowing analysts to inspect and revise mutation masks in real time—combining machine-driven automation with expert intuition to maximize vulnerability discovery efficiency.

3.2. Keyword byte awareness mechanism

The feature extraction in GDFuzz relies on deep neural networks (DNNs) and the interpretability analysis method LIME (Local Interpretable Model-agnostic Explanations). LIME, proposed by Ribeiro et al. (2016), is a model-agnostic interpretability method designed to explain the prediction behavior of any complex model on an individual input. It works by generating perturbed data points in the vicinity of the input being explained, approximating the model locally using a simpler surrogate model (e.g., linear regression), and computing feature importance to reveal the contributions of input features to the prediction result.

Specifically, GDFuzz first trains a binary classification model $f(i) \rightarrow \{0, 1\}$ to predict whether an input i can reach the target code region. Subsequently, GDFuzz employs LIME to perform a quantitative analysis of the contributions of input features to the classification model's predictions. This process generates a Feature Importance Map (FIM). Let x represent an input sample, n denotes its length (number of bytes), and $FIM(x)$ be the feature importance map for x . The FIM assigns an importance score s_i to each byte x_i of the input, where $i \in \{1, 2, 3, \dots, n\}$. Formally, the feature importance map is defined as:

$$FIM(x) = \{(x_1, s_1), (x_2, s_2), (x_3, s_3), \dots, (x_n, s_n)\} \quad (1)$$

Here, s_i represents the importance score of byte x_i , as computed by LIME, quantifying its contribution to the prediction outcome of the classification model. Using the FIM, GDFuzz identifies the importance scores s_i of each byte x_i in the input sample x . Bytes with higher importance scores are considered critical for reaching the target code region and are thus protected from mutation. Conversely, bytes with lower importance scores are deemed less significant and are prioritized for mutation.

The mutation mask $M(x)$ for a sample x is defined as:

$$M(x) = \{m_i \mid m_i = f(s_i), i \in \{1, 2, \dots, n\}\} \quad (2)$$

where m_i is a binary flag for each byte x_i , and $f(s_i)$ is a threshold-based function that assigns:

$$f(s_i) = \begin{cases} 1, & \text{if } s_i \geq \tau \\ 0, & \text{if } s_i < \tau \end{cases} \quad (3)$$

Here, τ is a predefined threshold determined experimentally or heuristically. This mutation mask ensures that the mutation strategy focuses on less critical bytes while preserving the structural integrity and semantic relevance of the input, thereby improving the efficiency and effectiveness of fuzzing.

As shown in Fig. 3, the feature extraction module successfully identifies that the keyword *ELEMENT* strongly correlates with the target

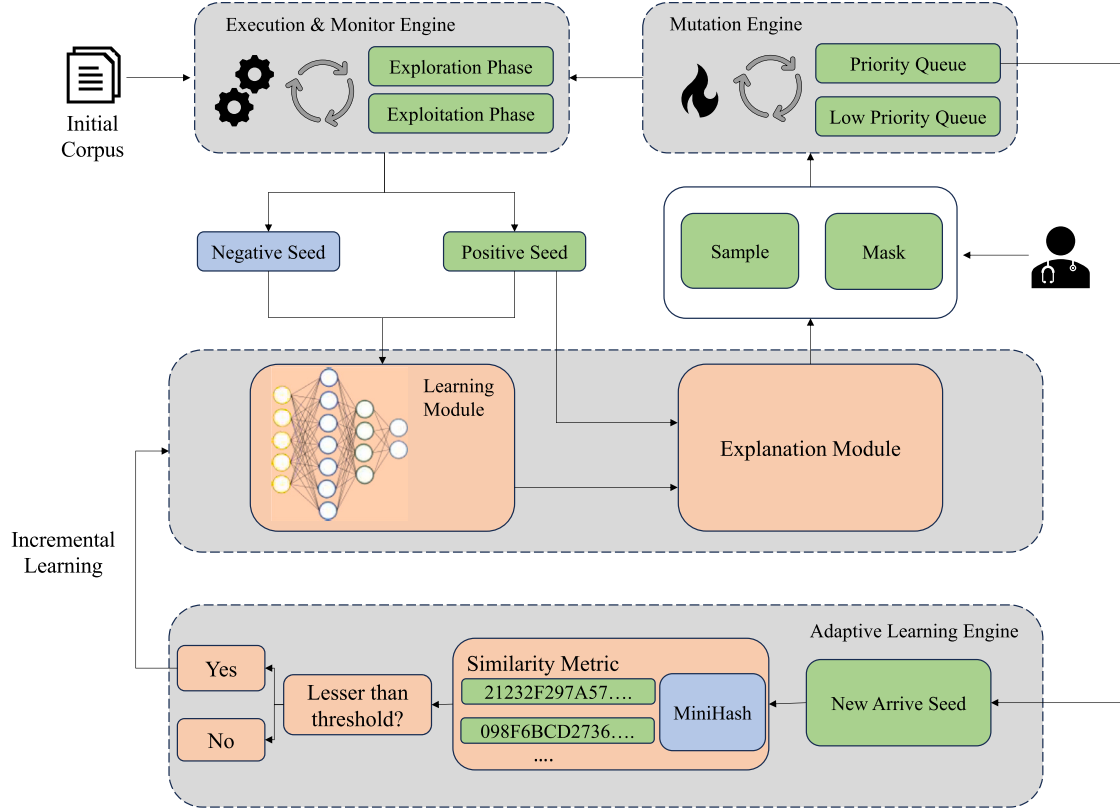


Fig. 2. An overview of GDFuzz.

```

<!DOCTYPE doc[
<!ELEMENT doc (src | dest)*>
<!ELEMENT src EMPTY>
<!ELEMENT dest EMPTY>
<!ALLIST src ref IDREF #IMPLIED>
<!ALLIST dest id ID #IMPLIED>
]>
<doc>
  <src ref="foo"/>
  <dest id='ftitle' &ampc ref==
</doc>

```

Fig. 3. Sample Mask of libxml2.

code region. This enables GDFuzz to protect the corresponding bytes from mutation, thereby increasing the likelihood of reaching and re-triggering the target path during subsequent executions. The input depicted in the figure was originally derived by mutating an initial seed provided by the libxml2 test suite, which is a typical reached seed that has already triggered the target execution at least once. GDFuzz uses this information to preserve critical input features and evolve the seed toward exploring more complex and meaningful execution paths.

Specifically, the mutation algorithm utilizing the sample mask is detailed in Algorithm 1. For each input seed s in the queue Q , we first retrieve its position index via the function `get_index(s)`. This index is used to access the corresponding sample mask $M[index]$ from the Feature Importance Map (FIM), which indicates the byte-level importance of the input. A mutation strategy is then randomly selected, such as bit flipping or byte substitution.

If the model training has been completed, the algorithm randomly selects a byte offset i from the range $[0, s.length)$ using the `random_choice(s.length)` operation. The mutation is only performed if the corresponding mask bit $M[index][i]$ equals 1, indicating that this byte is considered semantically relevant to the target code. In contrast, during the training phase, mutations are applied more freely across all byte positions to allow broader input space exploration.

By integrating the sample mask in this way, the mutation process maintains a balance between preserving key byte sequences and exploring new input variations, leading to a more focused fuzzing procedure.

Algorithm 1 Adaptive Mutation with Sample Masks

Require: Seed queue Q , Sample masks M , Model status $train_finish$

Ensure: Mutated seeds for testing

```

1: Initialize mutated_queue  $\leftarrow \emptyset$ 
2: for each seed  $s \in Q$  do
3:    $index \leftarrow get\_index(s)$ 
4:    $mutation\_strategy \leftarrow random\_strategy()$ 
5:   if  $train\_finish$  then
6:      $i \leftarrow random\_choice(s.length)$ 
7:     if  $M[index][i] == 0$  then
8:        $s[i] \leftarrow mutate(s[i], mutation\_strategy)$ 
9:     end if
10:  else
11:     $i \leftarrow random\_choice(s.length)$ 
12:     $s[i] \leftarrow mutate(s[i], mutation\_strategy)$ 
13:  end if
14:  Add  $s$  to mutated_queue
15: end for
16: return mutated_queue

```

3.3. Human-driven correction mechanism

To address the limitations of existing mutation strategies that overlook human understanding and intuition of input samples, GDFuzz introduces a human correction mechanism. This mechanism leverages explainable neural networks to generate a Feature Importance Map (FIM) for each input sample. The FIM highlights the significance of each byte in triggering the target function, providing an intuitive representation of which bytes are critical to the program's behavior.

For users, this feedback unveils the internal structure of the current sample during the fuzzing process and offers a data-driven foundation for refining subsequent mutation strategies. Based on the FIM, users can examine the importance of specific bytes in real time and adjust mask configurations to meet the requirements of the target function. For instance, when dealing with inputs that have strict format constraints, users can decide, based on the explainability analysis, whether to protect certain bytes from being mutated. This ensures that the sample remains valid and avoids the inefficiencies caused by invalid inputs due to format mismatches.

The artificial intervention based on interpretability enables fuzzing not only to rely on automated feedback but also to harness expert knowledge for efficient, targeted testing. It bridges the gap between automated exploration and human expertise, significantly enhancing the precision and effectiveness of directed fuzzing.

```

1  LOCAL(boolean) get_sos (j_decompress_ptr
    cinfo)
2  ...
3  if (! cinfo->marker->saw_SOF)
4  ERREXIT(cinfo, JERR_SOS_NO_SOF);
5  ...
6  if (length != (n * 2 + 6) || n < 1 || n
    > MAX_COMPS_IN_SCAN)
7  ERREXIT(cinfo, JERR_BAD_LENGTH);
8  ...
9  for (ci = 0, compptr = cinfo->comp_info;
10 ci < cinfo->num_components && ci <
    MAX_COMPS_IN_SCAN;
11 ci++, compptr++) {
12 if (cc == compptr->component_id &&
    !cinfo->cur_comp_info[ci])
13     goto id_found;
14 }
15 ERREXIT1(cinfo, JERR_BAD_COMPONENT_ID,
    cc);
16 id_found:
17     // data leak here
18     ...

```

Listing 3: Code Snippet for libjpeg

Listing 3 presents a code snippet of a data leak vulnerability (CVE-2013-6629) in the libjpeg library, specifically within the logic handling the JPEG Start of Scan (SOS) marker. From a testing perspective, effective fuzzing should prioritize mutation operations on input fields closely related to this marker. As illustrated in Fig. 5 and Table 1, GDFuzz's byte-awareness mechanism successfully identifies and preserves key bytes such as the SOS marker, the Start of Image (SOI) marker, and other relevant components that are crucial for triggering the target code. This identification is achieved automatically through explainable neural networks analysis, without requiring users to have prior knowledge of the file format or vulnerability details.

To further enhance flexibility and enable more targeted testing, GDFuzz provides users with visual feedback that highlights which parts of the input are most influential in reaching the target code. Based on this feedback, users can choose to refine the sample mask interactively. For example, they could decide to protect all bytes except those related

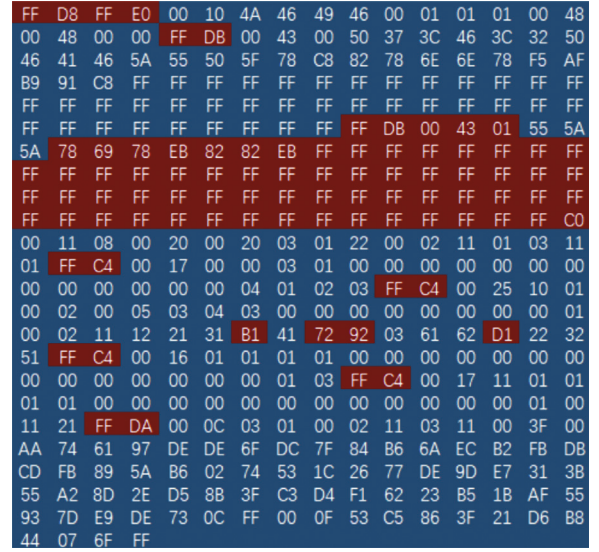


Fig. 4. Sample Mask of libjpeg.

Table 1

Recognition results of typical JPEG header fields in Fig. 4.

Marker name	Hex value	Recognized by GDFuzz?
Start of Image (SOI)	FF D8	✓
APP0 marker	FF E0	✓
Define Quantization Table (DQT)	FF DB	✓
Start of Frame (SOF0)	FF C0	✓
Define Huffman Table (DHT)	FF C4	✓
Define Restart Interval (DRI)	FF DD	X
Start of Scan (SOS)	FF DA	✓
End of image (EOI)	FF D9	X

to the SOS marker, effectively narrowing the mutation scope to the most semantically meaningful regions. This real-time interaction allows users to guide the fuzzing process based on intuitive understanding. It not only improves alignment between the fuzzing strategy and the target behavior but also reduces unnecessary exploration.

3.4. Dual-queue cyclic simulated annealing algorithm

The Dual-Queue Cyclic Simulated Annealing Algorithm (DQCSA) is an integral component of GDFuzz, aimed at addressing the inherent imbalance between the exploration and exploitation phases in traditional directed fuzzing frameworks. The detailed steps of the algorithm are shown in Algorithm 2. DQCSA introduces a novel design that partitions the sample pool into two distinct queues: the Exploration Queue, which holds all generated samples, and the Exploitation Queue, which retains samples that successfully reach the target area. This separation enables a clear functional distinction, facilitating both a broad exploration of the program's state space and focused testing on paths associated with potential vulnerabilities.

DQCSA dynamically balances these two queues through a cyclic scheduling mechanism informed by simulated annealing principles. At the start of the fuzzing process, the Exploration Queue is prioritized to maximize the discovery of new execution paths and behaviors. As the fuzzing process progresses and the diversity of generated samples grows, the priority gradually shifts toward the Exploitation Queue. This shift enhances testing on paths that are more likely to lead to the target. The transition is governed by an adaptive annealing schedule, which adjusts the queue-switching behavior in response to real-time feedback, such as the rate of discovering new paths or the distance to the target code.

A key aspect of DQCSA is its adaptive reset mechanism, which recalibrates the annealing schedule whenever a new reaching sample is identified in the Exploration Queue. The identification of such a sample often signals the discovery of a new path to the target area, providing valuable insights into previously unexplored control flow or data flow properties. By resetting the annealing temperature upon such discoveries, the algorithm redirects efforts toward exploration, ensuring that the newly uncovered features are adequately examined. This dynamic adjustment enhances the exploration of both control flow and data flow features critical to triggering vulnerabilities, thereby increasing the diversity of target-relevant samples.

By integrating these mechanisms, DQCSA maintains a dynamic equilibrium between exploration and exploitation, allowing for comprehensive program testing without compromising depth. This design not only improves the efficiency of fuzzing for complex vulnerabilities but also addresses the intricate interactions between control flow and data flow requirements. As such, DQCSA represents a methodical step forward in achieving a balance between broad exploratory coverage and precise, directed testing, making it a central element of GDFuzz's approach.

Algorithm 2 Dual-Queue Cyclic Simulated Annealing Algorithm (DQCSA)

Require: Initial seeds S_0 , Annealing parameters (τ_0, α) , Target code position T_{target}

- 1: Initialize Exploration Queue $Q_E \leftarrow S_0$
- 2: Initialize Exploitation Queue $Q_X \leftarrow \emptyset$
- 3: Set annealing temperature $\tau \leftarrow \tau_0$
- 4: Testing time counter $k \leftarrow 0$
- 5: **while** Stopping criteria not met **do**
- 6: $p_{explore} \leftarrow \exp(-k/\tau)$
- 7: **if** $\text{Random}() < p_{explore}$ **then**
- 8: $s \leftarrow \text{Select}(Q_E)$
- 9: **else**
- 10: $s \leftarrow \text{Select}(Q_X)$
- 11: **end if**
- 12: $S' \leftarrow \text{Mutate}(s)$
- 13: **if** $\text{is_interesting}(S')$ **then**
- 14: **if** S' reaches T_{target} **then**
- 15: Add S' to Q_X
- 16: **else**
- 17: Add S' to Q_E
- 18: **end if**
- 19: **end if**
- 20: $\tau \leftarrow \alpha \cdot \tau$
- 21: $k \leftarrow (\text{currentTime} - \text{startTime})$
- 22: **if** New reaching sample $S' \in Q_X$ detected **then**
- 23: Reset temperature $\tau \leftarrow \tau_0$
- 24: **end if**
- 25: **end while**
- 26: **return** Q_X

3.5. Adaptive incremental learning

To effectively address the challenges posed by increased sample diversity during fuzz testing while minimizing computational overhead, GDFuzz integrates an Adaptive Incremental Learning mechanism. As shown in Algorithm 3, this approach leverages a combination of byte-level feature differentiation and Locality-Sensitive Hashing (LSH) (Jafari et al., 2021) to dynamically evaluate sample diversity and determine the necessity of incremental model updates. Specifically, for each sample in the current batch, a MinHash signature is computed to approximate its semantic characteristics. The algorithm then compares this signature against previously stored signatures using the Jaccard similarity. If a sample is deemed novel (i.e., its similarity is below the

threshold α), it is included in a temporary buffer of new samples and its signature is added to the global set.

After processing the batch, the algorithm calculates the proportion of new samples as a measure of diversity change. If this ratio exceeds a diversity threshold θ , it indicates a meaningful shift in the input distribution. In response, the model enters an incremental training phase, where the top- k most diverse new samples are selected to update the model. This selective retraining ensures the model adapts to distributional drift without incurring full retraining costs. Finally, updated sample masks are regenerated based on the retrained model to guide the subsequent mutation process. This adaptive mechanism ensures that GDFuzz strikes a balanced trade-off between accurate feature identification and efficient resource utilization, maintaining both testing accuracy and system performance over fuzzing sessions.

Algorithm 3 Adaptive Incremental Learning with MinHash

Require: Sample set S , Current model M , MinHash signatures H , Similarity threshold α , Diversity threshold θ , Incremental training batch size k

Ensure: Updated model M' , Updated sample masks M_{mask}

- 1: Initialize $\text{diversity_change} \leftarrow 0$, $\text{new_samples} \leftarrow \emptyset$
- 2: **for** each sample $s \in S$ **do**
- 3: $\text{minhash_signature} \leftarrow \text{compute_MinHash}(s)$
- 4: $\text{similarity} \leftarrow \max\{\text{Jaccard}(H[i], \text{minhash_signature}) \mid H[i] \in H\}$
- 5: **if** $\text{similarity} < \alpha$ **then**
- 6: Add s to new_samples
- 7: Add minhash_signature to H
- 8: **end if**
- 9: **end for**
- 10: $\text{diversity_change} \leftarrow \frac{|\text{new_samples}|}{|S|}$
- 11: **if** $\text{diversity_change} > \theta$ **then**
- 12: $\text{training_batch} \leftarrow \text{select_top_k}(\text{new_samples}, k)$
- 13: $M' \leftarrow \text{incremental_train}(M, \text{training_batch})$
- 14: $M_{mask} \leftarrow \text{generate_sample_masks}(M')$
- 15: **else**
- 16: $M' \leftarrow M$
- 17: **end if**
- 18: **return** M', M_{mask}

4. Evaluation

To achieve a comprehensive evaluation of GDFuzz, this section primarily addresses the following research questions:

- **RQ1:** Compared to other state-of-the-art directed fuzzing tools, does GDFuzz achieve a higher number of effective executions for the same targets?
- **RQ2:** Does the Dual-Queue Cyclic Simulated Annealing algorithm effectively enhance the exploration at the target location?
- **RQ3:** Does the additional mechanism introduced by GDFuzz accelerate the vulnerability exposure process?
- **RQ4:** What is the performance overhead introduced by the additional modules in GDFuzz?

4.1. Experimental setup

Experimental environment. All experiments were conducted on a 64-bit Ubuntu 22.04 LTS system, equipped with an Intel Xeon Silver 4410Y processor, 128 GB RAM, and an NVIDIA RTX A6000 GPU with CUDA 12.2.

Dataset. We evaluated GDFuzz on both the standard Magma dataset (Hazimeh et al., 2021) and three additional real-world software applications. The detailed information on vulnerabilities, version numbers, and input types for these software applications is provided in Table 2.

The rationale for combining these two types of datasets is as follows:

Table 2
Vulnerability used in evaluation.

Program	Project	CVE ID	Input type
xmllint	libxml2	CVE-2017-9047(XML001)	xml
xmllint	libxml2	CVE-2017-7375(XML003)	xml
tiffcp	libtiff	CVE-2016-10270(TIF007)	tiff
pdftimages	poppler	CVE-2017-9865(PDF003)	pdf
pdftimages	poppler	CVE-2017-9776(PDF019)	pdf
tcpdump	tcpdump	CVE-2014-8768	pcap
tcpdump	tcpdump	CVE-2014-8769	pcap
djpeg	libjpeg	CVE-2013-6629	jpg
pcrtest	libpcrc	CVE-2017-7244	regex
xmllint	libxml2	CVE-2017-5969	xml

1. Magma serves as a representative evaluation dataset for general-purpose fuzzing tools, offering standard benchmarks.
2. The real-world software demonstrates GDFuzz’s capability in practical application scenarios and extends the range of input types to validate the generality and effectiveness of the byte-awareness mechanism.

Furthermore, the real-world software selected has been widely used in the evaluation of other advanced fuzzing tools (Fang et al., 2024; Luo et al., 2023), enhancing the representativeness and comprehensiveness of our dataset. To ensure that the evaluation aligns with our objectives, we adopted Magma’s construction methodology. Specifically, for the real-world software, we instrumented key locations and utilized shared memory to monitor execution feedback, enabling precise tracking of effective executions.

Baseline Fuzzers. To evaluate the effectiveness of GDFuzz, we selected several state-of-the-art coverage-guided fuzzing tools and directed fuzzing tools as baselines, including AFL, AFL++, AFLGo, SelectFuzz and FormatFuzz (Dutra et al., 2023). AFL was chosen as a reference point because most directed fuzzing tools are developed based on AFL. This makes AFL an excellent baseline for comparison in our evaluation. AFL++ is the most advanced coverage-guided fuzzing tool, incorporating recent innovations in fuzzing. It demonstrates superior performance and serves as a strong benchmark for coverage-guided fuzzers. AFLGo, proposed by Böhme, was the first directed fuzzing tool and represents a foundational benchmark in the directed fuzzing domain. SelectFuzz, a more recent directed fuzzing tool, improves upon AFLGo’s distance calculation algorithm and reflects the latest advancements in directed fuzzing techniques. These selected baseline fuzzing tools represent established standards in their respective domains, enabling a comprehensive evaluation of GDFuzz’s capabilities across both coverage-guided and directed fuzzing scenarios. In addition, we included FormatFuzzer, a recent grammar-aware fuzzer that emphasizes input format validity. While it is not a directed fuzzer, it offers a meaningful comparison in terms of input structure preservation.

4.2. Effective executions (RQ 1)

Effective executions reflect the ability of a fuzzing tool to generate high-quality inputs that traverse deep execution paths and ultimately uncover vulnerabilities. To evaluate whether GDFuzz can automatically enhance the proportion of effective executions among the total number of executions during testing, we conducted experiments under the configurations outlined in Section 4.1. These experiments were conducted in a controlled environment with identical initial test cases, evaluating all targets in the dataset over a fixed duration of 3 h per target. During this phase, no manual intervention or adjustments to the sample mask were employed to ensure an unbiased evaluation of GDFuzz’s automated capabilities. Since FormatFuzzer is not designed for text-based formats such as XML and regular expressions, it did not produce test results for these input types.

The experimental results, summarized in Table 3, demonstrate a significant improvement in the proportion of effective executions achieved

by GDFuzz across all tested scenarios. Here, an effective execution is defined as an input that successfully reaches the specified target code region, as determined by the directed fuzzing configuration. This metric evaluates the fuzzer’s ability to guide execution toward the intended location. Notably, for CVE-2017-9047 (XML001), the results reveal that approximately 80% of regular inputs were rejected early in the execution path due to the program’s stringent input validation logic. Specifically, as illustrated in Listing 1, an if condition checks for the presence of the ELEMENT string; inputs failing the check were terminated immediately without further execution. GDFuzz, leveraging its byte-awareness mechanism, successfully identified the “ELEMENT” field as an important feature through interpretability analysis. This enabled it to apply a targeted protection strategy, preserving this feature during the mutation process while allowing other parts of the input to evolve dynamically.

This directed mutation strategy ensured the influencing key bytes remained intact, substantially increasing the likelihood of generating inputs reaching the target program location. Consequently, GDFuzz not only improved testing efficiency but also accelerated the process of vulnerability discovery, at least achieving up to an average 185% improvement in effective execution rate compared to baseline fuzzing tools.

Additionally, Fig. 5 illustrates the temporal trend of effective executions during the testing phase. Initially, before GDFuzz accumulates sufficient reaching samples, its performance in terms of effective executions is comparable to other tools. However, once the keyword-aware mechanism is triggered, enabling the dynamic extraction and application of sample masks, the effective execution rate rapidly surpasses that of baseline tools. This trend highlights the pivotal role of the byte-awareness mechanism in driving the observed improvements in execution efficiency.

4.3. Sample diversity (RQ 2)

The Dual-Queue Cyclic Simulated Annealing (DQCSA) algorithm is an important improvement in GDFuzz, designed to address the critical balance between exploration and exploitation in fuzzing. By integrating two distinct queues—a discovery queue for uncovering new execution paths and a refinement queue for enhancing inputs that reach the target—DQCSA aims to enhance the diversity of data flow and control flow coverage at the target location.

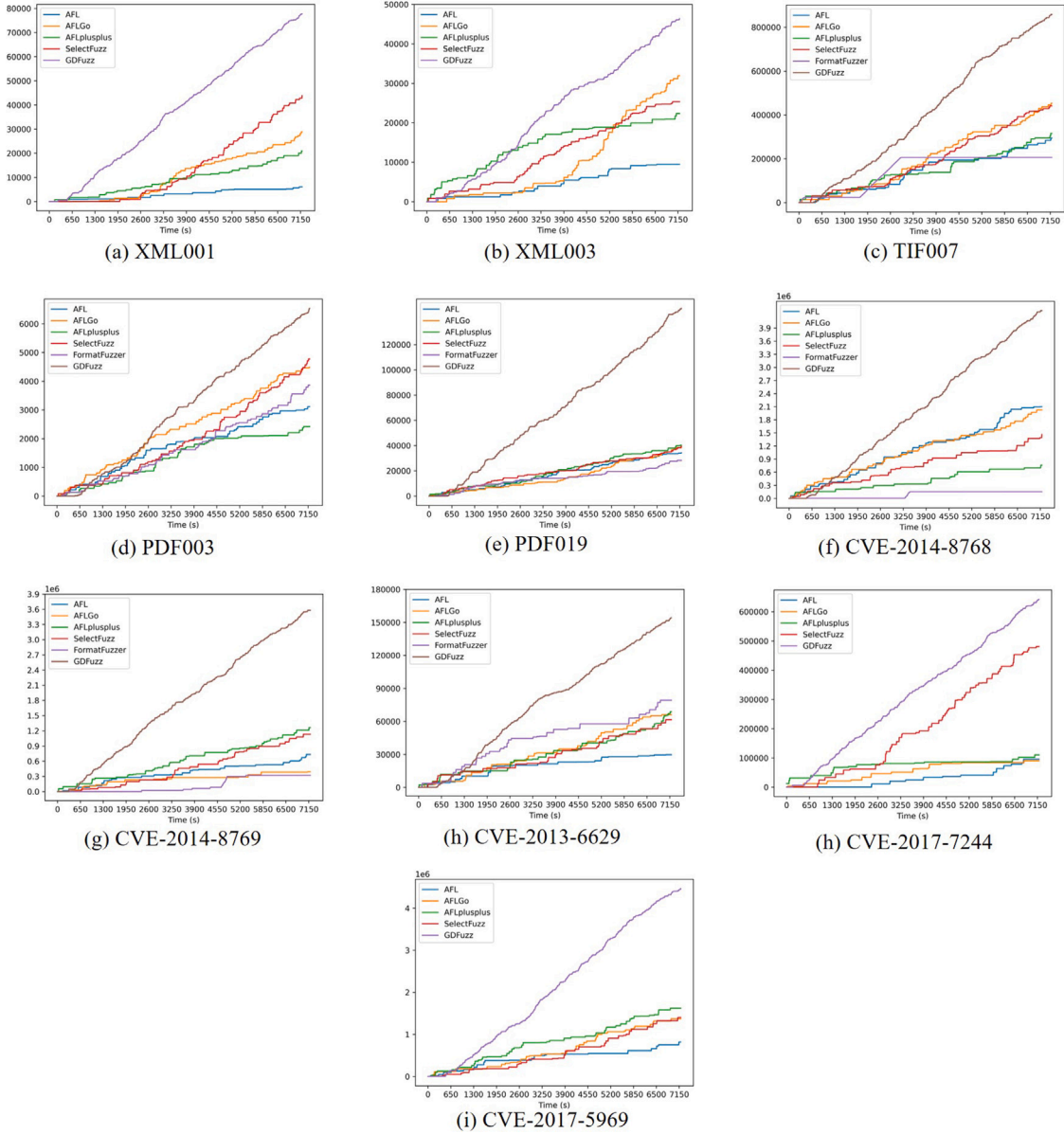
To rigorously evaluate the effectiveness of DQCSA, we designed a controlled experiment comparing three configurations: GDFuzz with DQCSA enabled (GDFuzz), GDFuzz without DQCSA (GDFuzz⁻), and the baseline directed fuzzing framework SelectFuzz. The core metric of interest was the diversity of execution paths leading to the target location, quantified through basic block coverage analysis. Specifically, we tracked and analyzed the unique control flow paths that preceded inputs reaching the target, allowing us to measure target-specific control flow diversity. This focus is motivated by the observation that vulnerability triggering typically requires not only reaching the target location but also satisfying certain data flow constraints—which are often influenced by the control flow taken en route to the target. Therefore, a more diverse set of control flows reaching the target increases the likelihood of encountering semantically distinct data states, thereby enhancing the chance of satisfying the conditions necessary to trigger the vulnerability. Based on this understanding, we regard it as a complementary and meaningful indicator of fuzzing effectiveness at the semantic level.

To isolate the diversity specific to the target region, we filtered execution traces to focus exclusively on the segments leading to the target basic block. This allowed us to identify variations in control flow paths and exclude inputs unrelated to meaningful exploration of the target. We further analyzed data flow diversity by inspecting the mutation patterns and the range of dynamic data states observed in inputs.

Table 3

The average effective executions rate in all fuzzers in 3 h.

CVE ID	AFL	AFL++	AFLGo	SelectFuzz	FormatFuzzer	GDFuzz	Up rate
CVE-2017-9047(XML001)	0.015986042	0.009589886	0.016196263	0.031780324	X	0.121939523	>283%
CVE-2017-7375(XML003)	0.001571310	0.003882001	0.007067401	0.005362996	X	0.015713152	>122%
CVE-2016-10270(TIF007)	0.342513355	0.341886630	0.607582208	0.520286127	0.179162370	0.892271457	>46%
CVE-2017-9865(PDF003)	0.000325172	0.000201632	0.000357459	0.000474791	0.000337482	0.001487289	>213%
CVE-2017-9776(PDF0019)	0.000855235	0.001017376	0.000969854	0.000890847	0.009388745	0.007586608	>645%
CVE-2014-8768	0.031307688	0.002260355	0.035576781	0.005295086	0.007282904	0.096926003	>172%
CVE-2014-8769	0.020885404	0.060555091	0.017907709	0.045991503	0.015089725	0.082356755	>36%
CVE-2013-6629	0.002252637	0.008396062	0.007821482	0.008246127	0.007599780	0.017284256	>105%
CVE-2017-7244	0.000157089	0.001949265	0.004563833	0.018363079	X	0.046291242	>152%
CVE-2017-5969	0.281007893	0.095398834	0.084272135	0.093713312	X	0.488167639	>73%

**Fig. 5.** Effective counts for baseline fuzzers and GDFuzz in 3 h.

Our findings, summarized in Table 4, indicate that GDFuzz without DQCSA achieved moderate improvements in target-specific sample diversity compared to SelectFuzz. This improvement, amounting to a 74% increase in unique control flow paths, can be attributed to GDFuzz's input format preservation and key-byte identification features. However, GDFuzz with DQCSA enabled outperformed both SelectFuzz and GDFuzz without DQCSA, average achieving 2.93x and 1.65x increases

in control flow diversity, respectively. This improvement highlights the effectiveness of DQCSA's cyclic annealing mechanism, which dynamically balances exploration and exploitation, facilitating more thorough exploration of distinct execution paths.

Interestingly, we observed negligible differences between GDFuzz with and without DQCSA in two test cases: CVE-2017-7244 and CVE-2017-5969. Both vulnerabilities are classified as shallow, with their

Table 4The average different path of SelectFuzz, GDFuzz with DQCSA enabled(GDFuzz) and GDFuzz without DQCSA(GDFuzz⁻).

CVE ID	SelectFuzz	GDFuzz ⁻	GDFuzz	Up rate	
				SelectFuzz	GDFuzz ⁻
CVE-2017-9047(XML001)	15	20	35	>2.33x	>1.75x
CVE-2017-7375(XML003)	87	102	159	>1.82x	>1.55x
CVE-2016-10270(TIF007)	13	25	28	>2.15x	>1.12x
CVE-2017-9865(PDF003)	9	11	27	>3.00x	>2.45x
CVE-2017-9776(PDF0019)	5	16	38	>7.60x	>2.37x
CVE-2014-8768	8	12	22	>2.75x	>1.83x
CVE-2014-8769	17	35	62	>3.64x	>1.77x
CVE-2013-6629	47	55	83	>1.76x	>1.5x
CVE-2017-7244	33	69	75	>2.27x	>1.08x
CVE-2017-5969	152	281	307	>2.01x	>1.09x
Avg. up rate				>2.93x	>1.65x

vulnerable functions located early in the program's call stack. This inherent characteristic limits the potential for control flow diversity, as fewer paths are available for exploration. Despite this limitation, GDFuzz with DQCSA still achieved a modest improvement, reinforcing its robustness across varying scenarios.

These findings underscore DQCSA's ability to enhance control and data flow diversity, particularly in complex scenarios where deep exploration is required. By dynamically prioritizing exploitation as the algorithm progresses, DQCSA effectively balances exploration and refinement, advancing the state of directed fuzzing techniques.

4.4. Vulnerability exposure speed (RQ 3)

The improvement in effective execution rate observed with GDFuzz is directly linked to its accelerated ability to uncover software vulnerabilities. We evaluated the performance consistency of GDFuzz under identical testing conditions, with all experiments conducted in a fully automated manner without human intervention. To assess the statistical reliability of its results, we applied the Mann-Whitney U test (Kim et al., 2024) to compare the Time-To-Exposure (TTE) distributions between GDFuzz and each baseline fuzzer across five independent runs for every target vulnerability. In each comparison, GDFuzz was selected as the reference group, and the null hypothesis assumed no significant difference in TTE distributions between GDFuzz and the corresponding fuzzer. The alternative hypothesis stated that a statistically significant difference exists. To account for multiple comparisons — one per baseline fuzzer, we applied the Benjamini-Hochberg procedure to control the false discovery rate (FDR).

As shown in Table 5, all uncorrected *p*-values were below the conventional significance threshold of 0.05. After applying the Benjamini-Hochberg correction, the adjusted critical *p*-value thresholds ranged from 0.008 to 0.05 depending on the number of comparisons per test case. In the majority of cases, GDFuzz outperformed other fuzzers, with corrected *p*-values falling within the significant range. On average, GDFuzz achieved a 2.3× improvement in vulnerability exposure speed compared to baseline approaches. Moreover, it consistently triggered crashes faster than other tools in most scenarios, confirming its strong performance and reliability in real-world applications.

A detailed analysis of individual test cases highlights both the strengths and nuances of GDFuzz's performance. For example, in the case of vulnerability CVE-2016-10270 (TIF003), the speedup was relatively modest. This outcome can be attributed to the simplicity of the vulnerability's trigger condition, which allowed baseline tools to expose the issue quickly using the initial seed inputs. However, even in this scenario, GDFuzz's interpretability module efficiently extracted the corresponding data format for the TIF images, ensuring a competitive execution rate. This indicates that while GDFuzz excels in scenarios requiring nuanced data format recognition and complex path exploration, its performance in simpler cases is more comparable to traditional tools.

Conversely, for vulnerabilities with intricate input requirements or deep control flow dependencies, GDFuzz demonstrated its unique

strengths. The integration of feature-aware mutation strategies allowed it to prioritize key input features effectively, enabling faster traversal of relevant execution paths. These results underscore GDFuzz's enhanced capability in reducing the time required to expose complex vulnerabilities, particularly in scenarios where traditional fuzzers struggle to adapt to specific input constraints.

In summary, the experimental findings illustrate GDFuzz's superior vulnerability exposure efficiency across diverse scenarios. By combining automated interpretability with feature-driven mutation, GDFuzz not only accelerates vulnerability detection but also ensures robustness in handling a variety of software testing challenges. Future work could explore extending this capability to more complex environments, such as distributed systems or multi-component applications, to further validate and refine the approach.

4.5. Overhead (RQ 4)

We analyzed the CPU performance overhead introduced by GDFuzz, as excessive overhead could hinder the fuzzing process by reducing the rate of test case generation and execution. To quantify this impact, GDFuzz was tested on three newly selected target programs for a duration of three hours, and its execution counts were compared to those of the fastest baseline fuzzing tool under identical conditions. The results indicate that GDFuzz incurs an average additional CPU overhead of 16%, primarily attributed to the processing logic required for generating and applying sample masks.

While this additional overhead might appear significant, it is accompanied by substantial gains in fuzzing efficiency. Specifically, GDFuzz achieved a 2.6× improvement in effective execution rate and a 2.3× acceleration in vulnerability exposure speed compared to the baseline. These results suggest that the slight reduction in execution throughput is outweighed by the significant improvements in test case quality and coverage.

GPU Resources. Unlike traditional fuzzing tools, GDFuzz utilizes GPU resources to accelerate the training of interpretable deep learning models, which play a critical role in identifying and preserving key sample formats during the fuzzing process. For this purpose, we employed an RTX A6000 GPU and a lightweight convolutional neural network (CNN) architecture. While this introduces additional GPU overhead, the training time per fuzzing execution cycle was minimal relative to the overall fuzzing process, making the cost negligible in practical terms.

Furthermore, leveraging GPU acceleration ensures that the interpretability model can be dynamically updated during fuzzing, maintaining its relevance across evolving execution contexts. This adaptability enhances the overall fuzzing performance while maintaining a manageable computational footprint.

As shown in Table 6, GDFuzz demonstrates a trade-off between resource utilization and fuzzing efficiency. The 16% of the average additional CPU overhead and negligible GPU cost are offset by measurable improvements in fuzzing outcomes, including a 2.6× increase in effective executions and a 2.3× speedup in vulnerability exposure.

Table 5

The average crash exposure time and the statistical test p values of the evaluated tools for each vulnerability.

Program	CVE ID	Vuln. code	AFL			AFL++			AFLGo			SelectFuzz			FormatFuzzer			GDFuzz*		
			TTE	p	Threshold	TTE	p	Threshold	TTE	p	Threshold	TTE	p	Threshold	TTE	p	Threshold	TTE	p	Threshold
xmllint	CVE-2017-9047(XML001)	valid.c:1331	3.05 h	0.013	0.030	3.28 h	0.019	0.040	2.49 h	0.007	0.010	2.09 h	0.009	0.020	X	–	–	0.70 h	1	0.050
xmllint	CVE-2017-7375(XML003)	parser.c:7994	0.22 h	0.007	0.030	0.21 h	0.004	0.010	0.18 h	0.006	0.020	0.17 h	0.008	0.040	X	–	–	0.12 h	1	0.050
tiffcp	CVE-2016-10270(TIF007)	tif_dirread.c:5995	0.09 h	0.014	0.041	0.17 h	0.008	0.033	0.16 h	0.006	0.025	0.18 h	0.004	0.016	0.48 h	0.003	0.008	0.10 h	1	0.050
poppler	CVE-2017-9865(PDF003)	ImageOutputDev.cc:409	1.87 h	0.007	0.042	2.49 h	0.005	0.033	1.62 h	0.003	0.018	1.17 h	0.003	0.008	1.92 h	0.004	0.025	0.80 h	1	0.050
poppler	CVE-2017-9776(PDF019)	JBIG2Stream.cc:810	T.O	–	–	20.48 h	0.008	0.038	18.75 h	0.006	0.030	12.01 h	0.002	0.012	T.O	–	–	8.41 h	1	0.050
tcpdump	CVE-2014-8768	print-geonet.c:244	0.13 h	0.018	0.033	0.14 h	0.032	0.042	0.12 h	0.003	0.008	0.11 h	0.007	0.020	0.08 h	0.016	0.025	0.09 h	1	0.050
tcpdump	CVE-2014-8769	print-udp.c:367	0.19 h	0.008	0.025	0.16 h	0.015	0.042	0.14 h	0.004	0.017	0.13 h	0.003	0.008	0.60 h	0.013	0.033	0.09 h	1	0.050
djpeg	CVE-2013-6629	jdmaker.c:357	0.05 h	0.003	0.025	0.04 h	0.002	0.008	0.04 h	0.007	0.042	0.04 h	0.006	0.033	0.02 h	0.002	0.016	0.01 h	1	0.050
pcrtest	CVE-2017-7244	pcrc_xclass.c:142	T.O	–	–	T.O	–	–	23.53 h	0.018	0.033	7.36 h	0.002	0.017	X	–	–	1.13 h	1	0.050
xmllint	CVE-2017-5969	valid.c:1191	3.37 h	0.004	0.030	2.36 h	0.003	0.020	2.41 h	0.001	0.010	1.85 h	0.007	0.040	X	–	–	1.48 h	1	0.050

GDFuzz*: Baseline in hypothesis testing.

p: p-value from the Mann-Whitney U test.

Threshold: Adjusted significance threshold after Benjamini-Hochberg correction.

Table 6

Execution counts of fastest fuzzer and GDFuzz.

CVE ID	Fastest fuzzer	GDFuzz	Overhead
sndfile_fuzzer	7.82M	6.80M	13%
xml_read_memory_fuzzer	28.6M	21.1M	26%
pdf_fuzzer	315k	286k	9%
Average			16%

These findings underscore that the introduced overhead is justified and manageable within the context of modern hardware capabilities, especially given the critical gains in testing accuracy and efficiency.

4.6. Case study: Human-guided fuzzing for CVE-2017-9047

In practical testing scenarios, automated fuzzing is the standard approach due to its efficiency and scalability. However, GDFuzz is designed to support human-in-the-loop intervention, enabling users to refine fuzzing strategies for more targeted vulnerability detection. To evaluate the effectiveness of this feature, we conducted a case study on the CVE-2017-9047 vulnerability, demonstrating how GDFuzz performs under human guidance.

CVE-2017-9047 pertains to a vulnerability in the XmlSprintfElementContent function, which is responsible for formatting XML element content into a buffer. The vulnerability arises when specific conditions related to the XML structure and the ELEMENT field are met, leading to buffer overflow. Fig. 6 illustrates one of the sample masks generated by GDFuzz during automated fuzzing. This mask effectively identifies and protects most format-critical bytes related to the XML structure, such as tag delimiters and attribute markers.

Upon analyzing the vulnerable function, users observed that mutations involving the ELEMENT field are most likely to trigger the vulnerability. Based on this insight, the sample mask was refined to prioritize mutations specifically on bytes associated with ELEMENT while preserving the overall XML structure. This refinement leverages human expertise to guide the fuzzing process, focusing computational resources on the most relevant input regions.

Experimental data reveals a significant improvement in vulnerability exposure speed when human guidance is introduced. Without human intervention, GDFuzz required 2534 s to trigger the vulnerability, whereas with human-guided adjustments, the triggering time was reduced to 1476 s—resulting in a 1.58× speed improvement. This acceleration highlights the effectiveness of incorporating human expertise into the fuzzing process. As shown in Fig. 6, the refined sample mask significantly narrowed the mutation scope, reducing irrelevant exploration and enhancing the likelihood of reaching the vulnerable code path.

4.7. Unknown vulnerability detection

To evaluate the capability of GDFuzz in discovering previously unknown vulnerabilities, we applied it to real-world software systems

on a relatively large scale. In total, GDFuzz was tested across 12 open-source software projects. Remarkably, as shown in Table 7, GDFuzz uncovered 8 previously undisclosed vulnerabilities, 4 of which have since been assigned CVE IDs. These findings are particularly notable given that the tested programs had already undergone extensive fuzzing campaigns by both the open-source community and well-established platforms such as OSS-Fuzz (Google, 2022).

One significant example involves a buffer overflow vulnerability in a widely used XML parser, where the vulnerability was buried deep within a rarely executed code path. GDFuzz successfully extracted the key input features necessary to traverse this path and triggered the vulnerability, a feat not achieved by baseline fuzzers during comparative testing. While this study focused on a selected set of programs, the promising results motivate large-scale testing across a broader range of open-source software projects, such as IoT devices, embedded systems, and distributed software.

5. Discussion

In this section, we focus on discussing the current limitations of GDFuzz and potential directions for future research.

Static Nature of Feature Interpretation. The core of GDFuzz's feature interpretation lies in leveraging neural networks and interpretability tools to extract key features in the input samples that are relevant to the target code paths. For programs with complex branching logic, such as those requiring multi-layered conditions to trigger specific paths, the interpretation may provide static results tailored to specific conditions but fail to dynamically adapt to varying path-triggering scenarios. To address this issue, we plan to incorporate a feedback loop into the fuzzing process. This loop will utilize samples that fail to trigger target paths to validate and adjust feature masks dynamically, thereby enhancing adaptability to diverse conditions.

Human-Guided Correction and Scalability Challenges. GDFuzz integrates a human-in-the-loop interface that enables users to refine mutation strategies based on the interpretability results generated by the system. However, this reliance on human expertise introduces practical limitations, particularly when users lack sufficient understanding of the target program or input format. In complex or large-scale testing scenarios, manual refinement may become inefficient or error-prone, thereby reducing the system's overall accessibility and usability. To address these challenges, we plan to incorporate large language models (LLMs) to further enhance the system's ability to assist users in interpreting input formats and identifying key features. Specifically, LLMs will be used to parse and explain structured input formats. They will highlight syntactic and semantic patterns relevant to the target code path and suggest candidate byte regions for protection or mutation based on natural language descriptions of the vulnerability context. This integration aims to guide users—especially those without deep technical expertise—toward more effective mask adjustments, narrowing the scope of required modifications and reducing the need for extensive prior knowledge.

Model Drift. While the current Adaptive Incremental Learning framework effectively supports directed fuzzing under typical conditions, we are investigating additional mechanisms to further extend

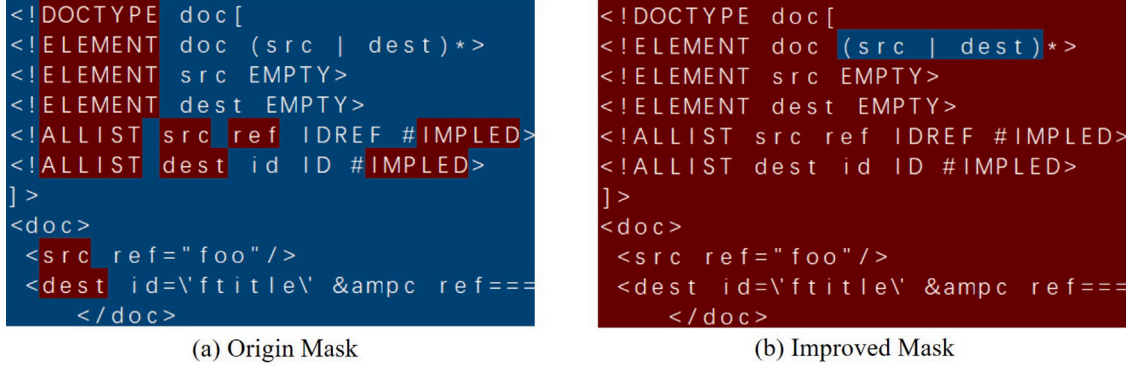


Fig. 6. Comparison before and after mask modification.

Table 7
New vulnerabilities detected by GDFuzz.

Project	Root case	Vul. type	Status	CVE
ffmpeg	crc.c:403	Assertion	Confirmed	CVE-2024-50851
tcpreplay	dlt_utils.c:58	seg. fault	Confirmed	CVE-2024-48522
ngiflib	ngiflib.c:852	Infinite loop	Confirmed	issue-31
ngiflib	ngiflib.c:556	Infinite loop	Confirmed	CVE-2023-37748
imgdataopt	imgdataopt.c:1100	seg. fault	Confirmed	CVE-2023-38778
lunasvg	lunasvg.cpp:232	seg. fault	Confirmed	issue-159
lunasvg	plutovg-ft-raster.c:889	Stack overflow	Confirmed	issue-157
fpng	fpng.cpp:3011	Heap overflow	Confirmed	issue-26

GDFuzz's capability in handling long-term execution scenarios. To further enhance GDFuzz's adaptability to long-running fuzzing sessions, we are exploring the integration of online concept drift detection methods into the system. These techniques, such as ADWIN and the Page-Hinkley detector, enable real-time monitoring of model prediction stability and feature relevance. By continuously analyzing indicators like classification confidence and mutation success rate, GDFuzz will be able to detect shifts in input semantics and update the Feature Importance Map (FIM) accordingly. This extension aims to improving its robustness and generalization capabilities in program behaviors.

By addressing these challenges, we aim to further enhance GDFuzz's adaptability, scalability, and usability in diverse and complex software testing scenarios.

6. Related work

In this section, we provide a concise summary of recent advancements in fuzz testing methodologies and the development trends of human-in-the-loop fuzz testing.

6.1. Directed fuzzing

In recent years, directed fuzzing has gained widespread attention for its ability to efficiently locate and reproduce vulnerabilities, particularly for 1-day vulnerabilities. Early work (Böhme et al., 2017) introduced the concept of Directed Greybox Fuzzing (DGF), using an energy scheduling strategy based on simulated annealing to allocate more resources to seeds closer to the target, laying the foundation for future research. However, its efficiency remains limited in complex program paths.

Subsequent improvements include Hawkeye (Chen et al., 2018), which dynamically adjusts seed prioritization and energy allocation, outperforming traditional fuzzers like AFL in real-world scenarios. BEACON (Huang et al., 2022) optimized path exploration by combining static analysis and runtime path pruning, reducing the number of paths without compromising target reachability. Windranger (Du et al., 2022) introduced Deviation Basic Blocks, improving seed distance calculation and mutation strategies, while SieveFuzz (Srivastava et al., 2022)

employed state constraints to guide testing resources toward target regions.

SelectFuzz (Luo et al., 2023) optimized fuzzing efficiency through selective path exploration based on control and data dependencies, while 1dFuzz (Yang et al., 2023) enhanced 1-day vulnerability detection via directed differential fuzzing, achieving significant performance gains.

Additionally, systems like TaintScope (Wang et al., 2010) and 1dVul (Peng et al., 2019) integrated taint analysis and symbolic execution to improve fuzzing specificity and efficiency, particularly for binary-level vulnerabilities.

Overall, these advancements highlight the evolution of directed fuzzing, focusing on optimizing path exploration, resource allocation, and integration with advanced analysis techniques to improve vulnerability detection and reproduction.

6.2. Improvement of Fuzzy testing based on neural networks

In recent years, neural network-based fuzz testing methods have achieved significant progress in software testing, primarily leveraging deep learning techniques to enhance the efficiency of test case generation and vulnerability detection. These methods integrate neural network models to intelligently guide the fuzzing process, thereby overcoming the limitations of traditional approaches. For example, GANFuzz framework, which employs Generative Adversarial Networks (GANs) to learn protocol syntax and generate test cases without relying on protocol specifications, making it applicable to both public and proprietary protocols and significantly improving testing efficiency (Hu et al., 2018). Similarly, NEUZZ (She et al., 2019), which uses a proxy neural network to smooth complex program branching behaviors and combines gradient-guided input generation strategies, further enhancing fuzzing efficiency.

In optimizing test case generation, multitask learning and attention mechanisms offer new directions for fuzz testing. For instance, the MTFuzz framework, which utilizes a multitask neural network to learn compact embeddings of the input space and guides mutation processes through high-gradient regions, thereby improving test specificity (She et al., 2020). Additionally, AMNeuzz (Zhang et al., 2024a),

which enhances NEUZZ's gradient strategies by incorporating attention mechanisms, achieving higher edge coverage within the same time constraints and further optimizing fuzzing performance.

Deep learning's predictive capabilities have also been effectively utilized to reduce invalid test cases. For example, FuzzGuard, which predicts the reachability of inputs using deep learning and filters out unreachable inputs before executing the target program, significantly improving the performance of directed gray-box fuzzing (Zong et al., 2020). This approach not only reduces computational resource waste but also enhances vulnerability detection efficiency.

Furthermore, the integration of reinforcement learning with fuzzing offers novel perspectives for test case generation. Konstantin Böttinger formalized fuzzing as a reinforcement learning problem and used a Deep Q-Learning algorithm to optimize test case generation, achieving more efficient vulnerability detection (Böttinger et al., 2018). Similarly, NeuFuzz, which leverages deep neural networks to guide intelligent seed selection, significantly improving vulnerability detection efficiency in gray-box fuzzing and discovering more vulnerabilities within shorter timeframes (Wang et al., 2019).

In intelligent seed generation, Skyfire, a framework that leverages knowledge from a large number of existing samples to generate well-distributed seed inputs, particularly suited for programs with highly structured inputs (Wang et al., 2017). This data-driven approach significantly enhances code coverage and vulnerability detection in fuzzing. Similarly, Learn&Fuzz, which combines neural networks and statistical machine learning techniques to automate the generation of input grammars for fuzzing and intelligently guides the fuzzing process using learned input probability distributions (Godefroid et al., 2017). While both methods highlight the importance of data-driven and machine learning approaches in seed generation, Skyfire focuses more on leveraging existing samples, whereas Learn&Fuzz emphasizes automated grammar generation.

In intelligent seed scheduling, MEUZZ, which employs supervised learning to achieve adaptive and generalizable seed scheduling, significantly enhancing the effectiveness of hybrid fuzzing (Chen et al., 2020).

6.3. Human in loop

Fuzz testing, as a critical automated software testing technique, has been widely adopted in security vulnerability detection. However, traditional fuzzing methods, relying on random input generation, often struggle to efficiently cover complex program paths. To address this, researchers have proposed various enhancements to improve fuzzing efficiency and coverage.

Bundt introduced a compartment analysis approach that identifies program compartments early, significantly improving coverage (Bundt et al., 2023). Demonstrated in the OSS-Fuzz project, this method offers a stable foundation for automation but shows limitations for highly dynamic programs. Maier and Seidel developed the JMPscare tool, enabling binary fuzzing through introspection techniques, thus extending fuzzing to scenarios lacking source code (Maier and Seidel, 2021). However, its adaptability to complex binaries remains uncertain.

In the realm of human-machine collaboration, Yan et al. (2023) proposed InFuzz, which employs visual bottleneck analysis and code annotations to enable users to intervene in the fuzzing process, highlighting the value of human collaboration (Yan et al., 2023). Fang introduced Dynamic Directed Gray-box Fuzzing (DDGF), utilizing path analysis to dynamically adjust testing directions and substantially improve efficiency, though its complexity limits broader application (Fang et al., 2024). Aschermann presented the Ijon mechanism, which leverages human annotations to guide fuzzing, successfully addressing challenges in the CGC(DARPA CGC project) dataset but facing scalability issues due to reliance on human analysts (Aschermann et al., 2020).

7. Conclusion

This study introduces a novel fuzz testing tool, GDFuzz, designed to address critical challenges in input generation and path exploration faced by traditional fuzz testing approaches. By incorporating interpretable neural networks and dynamic masking mechanisms, GDFuzz automates the extraction and protection of key input features, providing targeted support for input mutation. Additionally, the integration of a real-time intervention interface with visual feedback enables users to effectively optimize the fuzzing process, highlighting the advantages of human-machine collaboration.

Experimental results demonstrate that GDFuzz significantly outperforms existing fuzz testing tools in generating effective samples for multiple complex software applications. Its superior performance in target area coverage and sample effectiveness is particularly notable for software with strict format requirements, such as XML parsers and JPEG libraries. These findings validate the potential of feature-interpretation-driven mutation strategies and dynamic feedback mechanisms in enhancing fuzz testing efficiency.

However, GDFuzz has certain limitations. For instance, the static nature of feature extraction may impact its generalization capabilities in dynamic scenarios, and the dependency on user intervention could introduce additional operational complexity. These limitations highlight future research directions, such as dynamic feature modeling, further automation, and the development of a more universal framework adaptable to a broader range of software types.

In summary, GDFuzz offers an innovative fuzz testing approach that combines automation and human-machine collaboration, significantly improving the efficiency and effectiveness of testing complex software. It provides new insights and directions for the future development of fuzzing.

CRedit authorship contribution statement

Junru Li: Writing – original draft, Validation, Software, Methodology, Conceptualization. **Zhongxu Yin:** Writing – review & editing, Investigation, Data curation. **Guoxiao Zong:** Writing – review & editing. **Haiya Sang:** Writing – review & editing, Data curation.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author(s) used ChatGPT in order to improve language and readability. After using this tool/service, the author(s) reviewed and edited the content as needed and take(s) full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The data that has been used is confidential.

References

- Aschermann, C., Schumilo, S., Abbasi, A., Holz, T., 2020. Ijon: Exploring deep state spaces via fuzzing. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1597–1612.
- Böhme, M., Cadar, C., Roychoudhury, A., 2020. Fuzzing: Challenges and reflections. *IEEE Softw.* 38 (3), 79–86.
- Böhme, M., Pham, V.-T., Nguyen, M.-D., Roychoudhury, A., 2017. Directed greybox fuzzing. In: Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security. pp. 2329–2344.
- Böttinger, K., Godefroid, P., Singh, R., 2018. Deep reinforcement fuzzing. In: 2018 IEEE Security and Privacy Workshops. SPW, IEEE, pp. 116–122.
- Bundt, J., Fasano, A., Dolan-Gavitt, B., Robertson, W., Leek, T., 2023. Homo in machina: Improving fuzz testing coverage via compartment analysis. In: 2023 IEEE Conference on Software Testing, Verification and Validation. ICST, pp. 117–128. <http://dx.doi.org/10.1109/ICST57152.2023.00020>.
- Chen, Y., Ahmadi, M., Wang, B., Lu, L., et al., 2020. {MEUZZ}: Smart seed scheduling for hybrid fuzzing. In: 23rd International Symposium on Research in Attacks, Intrusions and Defenses. RAID 2020, pp. 77–92.
- Chen, H., Xue, Y., Li, Y., Chen, B., Xie, X., Wu, X., Liu, Y., 2018. Hawkeye: Towards a desired directed grey-box fuzzer. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 2095–2108.
- Du, Z., Li, Y., Liu, Y., Mao, B., 2022. Windranger: A directed greybox fuzzer driven by deviation basic blocks. In: Proceedings of the 44th International Conference on Software Engineering. pp. 2440–2451.
- Dutra, R., Gopinath, R., Zeller, A., 2023. Formatfuzzer: Effective fuzzing of binary file formats. *ACM Trans. Softw. Eng. Methodol.* 33 (2), 1–29.
- Fang, H., Zhang, K., Yu, D., Zhang, Y., 2024. DDGF: Dynamic directed greybox fuzzing with path profiling. In: Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 832–843.
- Godefroid, P., Peleg, H., Singh, R., 2017. Learn&fuzz: Machine learning for input fuzzing. In: 2017 32nd IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 50–59.
- Google, 2022. Oss-fuzz. <https://github.com/google/oss-fuzz>.
- Gulmez, S., Kakisim, A.G., Sogukpinar, I., 2024. XRank: Explainable deep learning-based ransomware detection using dynamic analysis. *Comput. Secur.* 139, 103703.
- Hazimeh, A., Herrera, A., Payer, M., 2021. Magma: A ground-truth fuzzing benchmark. In: Abstract Proceedings of the 2021 ACM SIGMETRICS/International Conference on Measurement and Modeling of Computer Systems. pp. 81–82.
- Hu, Z., Shi, J., Huang, Y., Xiong, J., Bu, X., 2018. GANFuzz: A GAN-based industrial network protocol fuzzing framework. In: Proceedings of the 15th ACM International Conference on Computing Frontiers. pp. 138–145.
- Huang, H., Guo, Y., Shi, Q., Yao, P., Wu, R., Zhang, C., 2022. Beacon: Directed grey-box fuzzing with provable path pruning. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 36–50.
- Jafari, O., Maurya, P., Nagarkar, P., Islam, K.M., Crushev, C., 2021. A survey on locality sensitive hashing algorithms and their applications. *arXiv preprint arXiv:2102.08942*.
- Kim, T.E., Choi, J., Im, S., Heo, K., Cha, S.K., 2024. Evaluating directed fuzzers: Are we heading in the right direction? *Proc. ACM Softw. Eng.* 1 (FSE), <http://dx.doi.org/10.1145/3643741>.
- Lee, G., Shim, W., Lee, B., 2021. Constraint-guided directed greybox fuzzing. In: 30th USENIX Security Symposium. USENIX Security 21, pp. 3559–3576.
- Liang, J., Jiang, Y., Wang, M., Jiao, X., Chen, Y., Song, H., Choo, K.-K.R., 2019. Deepfuzzer: Accelerated deep greybox fuzzing. *IEEE Trans. Dependable Secur. Comput.* 18 (6), 2675–2688.
- Lin, P., Zhou, Z., Zhang, S., 2020. An improved directed grey-box fuzzer. In: 2020 IEEE 9th Joint International Information Technology and Artificial Intelligence Conference. ITAIC, Vol. 9, pp. 1497–1502. <http://dx.doi.org/10.1109/ITAIC49862.2020.9338761>.
- Luo, C., Meng, W., Li, P., 2023. Selectfuzz: Efficient directed fuzzing with selective path exploration. In: 2023 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 2693–2707.
- Maier, D., Seidel, L., 2021. Jmpscare: Introspection for binary-only fuzzing. In: Workshop on Binary Analysis Research. BAR, Vol. 2021, p. 21.
- Marinescu, P.D., Cadar, C., 2013. KATCH: High-coverage testing of software patches. In: Proceedings of the 2013 9th Joint Meeting on Foundations of Software Engineering. pp. 235–245.
- OSS-Fuzz, 2022. Fuzz introspector. <https://github.com/ossf/fuzz-introspector>.
- Peng, J., Li, F., Liu, B., Xu, L., Liu, B., Chen, K., Huo, W., 2019. 1Dvul: Discovering 1-day vulnerabilities through binary patches. In: 2019 49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks. DSN, IEEE, pp. 605–616.
- Ribeiro, M.T., Singh, S., Guestrin, C., 2016. "Why Should I Trust You?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, San Francisco, CA, USA, August 13–17, 2016. pp. 1135–1144.
- She, D., Krishna, R., Yan, L., Jana, S., Ray, B., 2020. MTFuzz: fuzzing with a multi-task neural network. In: Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering. pp. 737–749.
- She, D., Pei, K., Epstein, D., Yang, J., Ray, B., Jana, S., 2019. Neuzz: Efficient fuzzing with neural program smoothing. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 803–817.
- Shi, J., Wang, Z., Feng, Z., Lan, Y., Qin, S., You, W., Zou, W., Payer, M., Zhang, C., 2023. {AifoRE}: Smart fuzzing based on automatic input format reverse engineering. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 4967–4984.
- Shi, J., Zou, W., Zhang, C., Tan, L., Zou, Y., Peng, Y., Huo, W., 2022. CAMFuzz: Explainable fuzzing with local interpretation. *Cybersecurity* 5 (1), 17.
- Srivastava, P., Nagy, S., Hicks, M., Bianchi, A., Payer, M., 2022. One fuzz doesn't fit all: Optimizing directed fuzzing via target-tailored program state restriction. In: Proceedings of the 38th Annual Computer Security Applications Conference. pp. 388–399.
- Wan, Y., Wang, W., Sun, J., Tian, D., 2024. Directed fuzzing based on bottleneck detection. In: Proceedings of the 2024 5th International Conference on Computing, Networks and Internet of Things. pp. 32–37.
- Wang, J., Chen, B., Wei, L., Liu, Y., 2017. Skyfire: Data-driven seed generation for fuzzing. In: 2017 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 579–594.
- Wang, T., Wei, T., Gu, G., Zou, W., 2010. TaintScope: A checksum-aware directed fuzzing tool for automatic software vulnerability detection. In: 2010 IEEE Symposium on Security and Privacy. IEEE, pp. 497–512.
- Wang, Y., Wu, Z., Wei, Q., Wang, Q., 2019. Neufuzz: Efficient fuzzing with deep neural network. *IEEE Access* 7, 36340–36352.
- Wong, F., Zheng, E.J., Valeri, J.A., Donghia, N.M., Anahtar, M.N., Omori, S., Li, A., Cubillos-Ruiz, A., Krishnan, A., Jin, W., et al., 2024. Discovery of a structural class of antibiotics with explainable deep learning. *Nature* 626 (7997), 177–185.
- Xiang, Y., Zhang, X., Liu, P., Ji, S., Liang, H., Xu, J., Wang, W., 2024. Critical code guided directed greybox fuzzing for commits. In: 33rd USENIX Security Symposium. USENIX Security 24, pp. 2459–2474.
- Yan, Q., Cao, H., Lu, S., Huang, M., 2023. InFuzz: An interactive tool for enhancing efficiency in fuzzing through visual bottleneck analysis (registered report). In: Proceedings of the 2nd International Fuzzing Workshop. pp. 56–61.
- Yang, S., He, Y., Chen, K., Ma, Z., Luo, X., Xie, Y., Chen, J., Zhang, C., 2023. 1dFuzz: Reproduce 1-day vulnerabilities with directed differential fuzzing. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 867–879.
- Yun, I., Lee, S., Xu, M., Jang, Y., Kim, T., 2018. {QsYM}: A practical concolic execution engine tailored for hybrid fuzzing. In: 27th USENIX Security Symposium. USENIX Security 18, pp. 745–761.
- Zakeri Nasrabadi, M., Parsa, S., Kalae, A., 2021. Format-aware learn&fuzz: deep test data generation for efficient fuzzing. *Neural Comput. Appl.* 33 (5), 1497–1513.
- Zhang, Y., Liu, Y., Xu, J., Wang, Y., 2024b. Predecessor-aware directed greybox fuzzing. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 1884–1900. <http://dx.doi.org/10.1109/SP54263.2024.00040>.
- Zhang, L., Wang, B., Liu, C., Wen, M., Zhang, Y., Wang, L., 2024a. AMNeuzz: A strongly directed fuzz testing method based on attention mechanism. In: Proceedings of the 2024 10th International Conference on Computing and Data Engineering. pp. 105–110.
- Zhou, C., Wang, M., Liang, J., Liu, Z., Sun, C., Jiang, Y., 2019. Visfuzz: Understanding and intervening fuzzing with interactive visualization. In: 2019 34th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 1078–1081.
- Zhu, S., Wang, J., Sun, J., Yang, J., Lin, X., Wang, T., Zhang, L., Cheng, P., 2023. Better pay attention whilst fuzzing. *IEEE Trans. Softw. Eng.*
- Zong, P., Lv, T., Wang, D., Deng, Z., Liang, R., Chen, K., 2020. {FuzzGuard}: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning. In: 29th USENIX Security Symposium. USENIX Security 20, pp. 2255–2269.